

Министерство образования и науки РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Омский государственный технический университет»

Кафедра «Информатика и вычислительная техника»

Отчёт по расчётно-графической работе
по дисциплине
«Проектирование и тестирование программного обеспечения»

Выполнил

Студент гр. ИИ-221

Нурсалимов Д.Е. 18.05.24

(подпись, дата)



Проверил

Ст. пр. Звонов А. О 18.05.24

(подпись, дата)



Омск 2024

Программа считывает входные правила. После ввода фактов программа анализирует их и выводит результат в виде выполняемого правила.

Была разработана UML-диаграмма, предусматривающая работу с правилами (рис. 1):

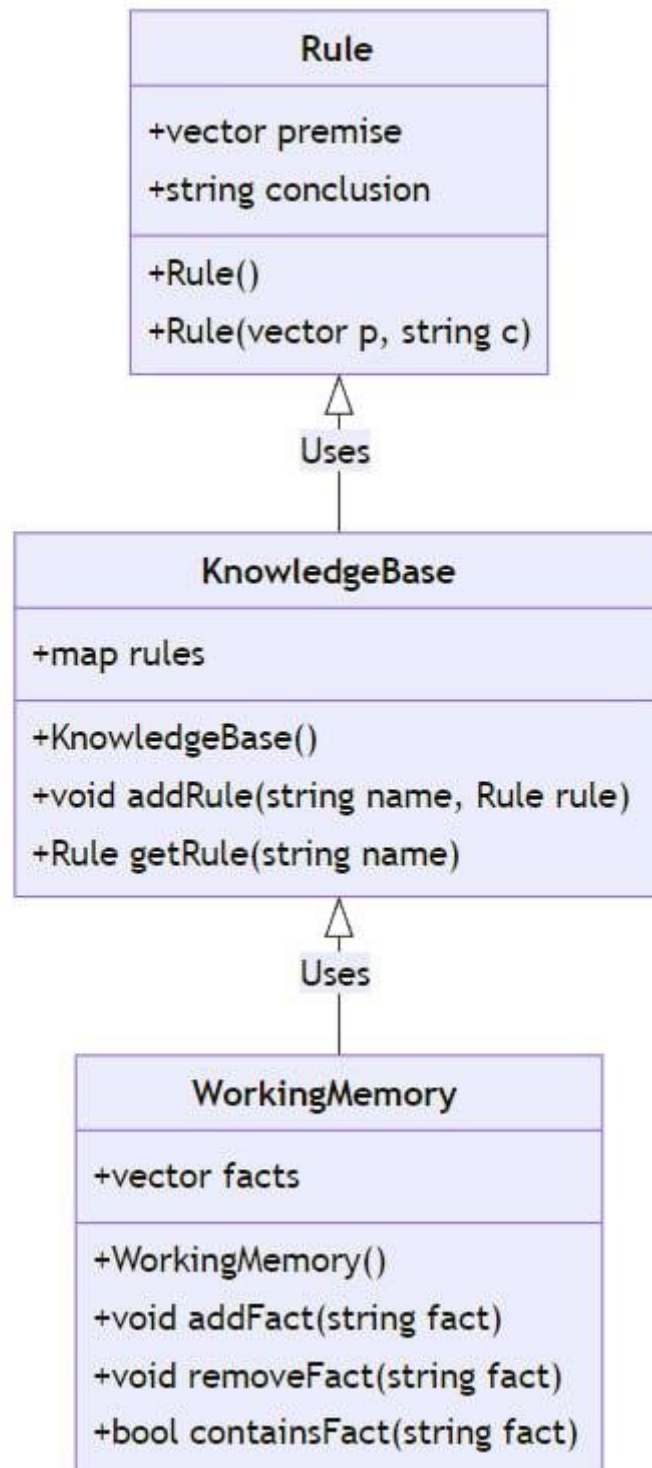


Рисунок 1 – UML диаграмма

Код программы, написанный на языке программирования C++:

```
#include <iostream>
#include <vector>
#include <string>
#include <map> // Включает ассоциативный массив (map)

using namespace std;

// Класс для представления правила
class Rule {
public:
    vector<string> premise; // Вектор для хранения посылок правила
    string conclusion; // Строка для хранения заключения правила
    // Конструкторы
    Rule() {} // Пустой конструктор
    Rule(vector<string> p, string c) : premise(p), conclusion(c) {} // Конструктор с
параметрами
};

// Класс для представления базы знаний
class KnowledgeBase {
public:
    map<string, Rule> rules; // Ассоциативный массив для хранения правил
    KnowledgeBase() {} // Пустой конструктор
    // Метод для добавления правила в базу знаний
    void addRule(string name, Rule rule) {
        rules[name] = rule; // Добавление правила в map по имени правила
    }
    // Метод для получения правила по имени
    Rule getRule(string name) {
        return rules[name]; // Возвращение правила по имени
    }
};

// Класс для представления рабочей памяти
class WorkingMemory {
public:
    vector<string> facts; // Вектор для хранения фактов
    WorkingMemory() {} // Пустой конструктор
    // Метод для добавления факта в рабочую память
    void addFact(string fact) {
        facts.push_back(fact); // Добавление факта в вектор фактов
    }
    // Метод для удаления факта из рабочей памяти
    void removeFact(string fact) {
        for (int i = 0; i < facts.size(); i++) { // Проход по всем фактам
            if (facts[i] == fact) { // Проверка совпадения факта
                facts.erase(facts.begin() + i); // Удаление факта из вектора
                break; // Выход из цикла после удаления
            }
        }
    }
    // Метод для проверки наличия факта в рабочей памяти
    bool containsFact(string fact) {
        for (string f : facts) { // Проход по всем фактам
            if (f == fact) { // Проверка совпадения факта
                return true; // Возврат true, если факт найден
            }
        }
        return false; // Возврат false, если факт не найден
    }
};

int main() {
```

```

    setlocale(LC_ALL, "RUS"); // Установка локали для корректного вывода на русском
языке
    WorkingMemory workingMemory; // Создание объекта рабочей памяти
    KnowledgeBase knowledgeBase; // Создание объекта базы знаний

    // Загрузка правил
    int numRules; // Переменная для хранения количества правил
    cout << "Введите количество правил: "; // Вывод запроса на ввод количества
правил
    cin >> numRules; // Ввод количества правил
    for (int i = 0; i < numRules; i++) { // Цикл для ввода правил
        string name, premise, conclusion; // Переменные для хранения имени, посылки
и заключения правила
        cout << "Введите имя правила: "; // Вывод запроса на ввод имени правила
        cin >> name; // Ввод имени правила
        cout << "Введите посылки правила: "; // Вывод запроса на ввод посылки
правила
        cin >> premise; // Ввод посылки правила
        cout << "Введите заключение правила: "; // Вывод запроса на ввод заключения
правила
        cin >> conclusion; // Ввод заключения правила
        vector<string> premiseVector; // Создание вектора для хранения посылки
        premiseVector.push_back(premise); // Добавление посылки в вектор
        Rule rule(premiseVector, conclusion); // Создание объекта правила
        knowledgeBase.addRule(name, rule); // Добавление правила в базу знаний
    }

    // Ввод фактов
    int numFacts; // Переменная для хранения количества фактов
    cout << "Введите количество фактов: "; // Вывод запроса на ввод количества
фактов
    cin >> numFacts; // Ввод количества фактов
    for (int i = 0; i < numFacts; i++) { // Цикл для ввода фактов
        string fact; // Переменная для хранения факта
        cout << "Введите факт: "; // Вывод запроса на ввод факта
        cin >> fact; // Ввод факта
        workingMemory.addFact(fact); // Добавление факта в рабочую память
    }

    // Цикл выполнения правил
    while (true) { // Бесконечный цикл для выполнения правил
        bool ruleFired = false; // Флаг, указывающий, выполнено ли хотя бы одно
правило
        for (auto it = knowledgeBase.rules.begin(); it != knowledgeBase.rules.end();
it++) { // Цикл по всем правилам в базе знаний
            Rule rule = it->second; // Получение правила из базы знаний
            bool premiseMatched = true; // Флаг, указывающий, все ли посылки правила
выполнены
            for (string premise : rule.premise) { // Цикл по всем посылкам правила
                if (!workingMemory.containsFact(premise)) { // Проверка, выполнена
ли посылка
                    premiseMatched = false; // Сброс флага, если посылка не
выполнена
                    break; // Выход из цикла по посылкам
                }
            }
            if (premiseMatched) { // Проверка, выполнены ли все посылки
                cout << "Выполнено правило: " << it->first << endl; // Вывод
сообщения о выполнении правила
                workingMemory.addFact(rule.conclusion); // Добавление заключения
правила в рабочую память
                ruleFired = true; // Установка флага, что правило выполнено
                break; // Выход из цикла по правилам
            }
        }
    }
}

```

```
        if (!ruleFired) { // Проверка, выполнено ли хотя бы одно правило
            cout << "Правила не выполнены." << endl; // Вывод сообщения о том, что
правила не выполнены
            break; // Выход из цикла while
        }
    }

    return 0;
}
```

После успешно выполненных операций над исходными данными пользователю выводится результат.

Вывод результатов программы:

```
Введите количество правил: 2
Введите имя правила: 1
Введите посылки правила: светит_солнце
Введите заключение правила: можно_гулять
Введите имя правила:
2
Введите посылки правила: идет_дождь
Введите заключение правила: нельзя_гулять
Введите количество фактов: 2
Введите факт: светит_солнце
Введите факт: не_идет_дождь
```

Выполнено правило: 1

```
C:\Users\Данияр\Desktop\Prn x + v
Введите количество правил: 2
Введите имя правила: 1
Введите посылки правила: я_голоден
Введите заключение правила: можно_покушать
Введите имя правила: 2
Введите посылки правила: я_сытый
Введите заключение правила: можно_погулять
Введите количество фактов: 2
Введите факт: я_голоден
Введите факт: я_не_сытый
```

Выполнено правило: 1

Рисунок 2 – Вывод тестовых сценариев программы

Вывод

В ходе работы были написаны программы на языке программирования C++. Программа решила поставленную задачу и прошла проверку при стандартном тестировании, а также при тестировании с некорректными (по условию задачи) входными данными.